40009 31

Haskell Sequences

Submitters

---

kw1425

Emarking

```
Final Tests: Summary for kw1425 of c1
PPT 21
-------------------------------------

  Public Tests:
    student-tests/tests/maxOf2:              3 / 3
    student-tests/tests/maxOf3:              3 / 3
    student-tests/tests/isADigit:            2 / 2
    student-tests/tests/isAlpha:             2 / 2
    student-tests/tests/digitToInt:          2 / 2
    student-tests/tests/toUpper:             2 / 2
    student-tests/tests/arithmeticSeq:       4 / 4
    student-tests/tests/geometricSeq:        4 / 4
    student-tests/tests/arithmeticSeries:    4 / 4
    student-tests/tests/geometricSeries:     4 / 4
    original-tests/tests/maxOf2:             3 / 3
    original-tests/tests/maxOf3:             3 / 3
    original-tests/tests/isADigit:           2 / 2
    original-tests/tests/isAlpha:            2 / 2
    original-tests/tests/digitToInt:         2 / 2
    original-tests/tests/toUpper:            2 / 2
    original-tests/tests/arithmeticSeq:      4 / 4
    original-tests/tests/geometricSeq:       4 / 4
    original-tests/tests/arithmeticSeries:   4 / 4
    original-tests/tests/geometricSeries:    4 / 4

Git Repo: git@gitlab.doc.ic.ac.uk:lab2526_autumn/haskellsequences_kw1425.git
Commit ID: dccd7
Git Diff: https://gitlab.doc.ic.ac.uk/lab2526_autumn/haskellsequences_kw1425/-/compare/master..dccd7?from_project_id=128588
```

```haskell
module Sequences where

import Data.Char (ord, chr)

-- | Returns the first argument if it is larger than the second,
-- the second argument otherwise
maxOf2 :: Int -> Int -> Int
maxOf2 x y
    | x > y = x
    | otherwise = y

-- | Returns the largest of three Ints
maxOf3 :: Int -> Int -> Int -> Int
maxOf3 x y z = maxOf2 (maxOf2 x y) (maxOf2 y z)

-- | Returns True if the character represents a digit '0'..'9';
-- False otherwise
isADigit :: Char -> Bool
isADigit x
    | ord x > 47 && ord x < 58  = True
    | otherwise = False

-- | Returns True if the character represents an alphabetic
-- character either in the range 'a'..'z' or in the range 'A'..'Z';
-- False otherwise
isAlpha :: Char -> Bool
isAlpha x
    | (ord x > 96 && ord x < 123) || (ord x > 64 && ord x < 91) = True
    | otherwise = False

-- | Returns the integer [0..9] corresponding to the given character.
-- Note: this is a simpler version of digitToInt in module Data.Char,
-- which does not assume the precondition.
digitToInt :: Char -> Int
-- Pre: the character is one of '0'..'9'
digitToInt x = ord x - 48

-- | Returns the upper case character corresponding to the input
-- where appropriate.
toUpper :: Char -> Char
toUpper x
    | ord x > 96 && ord x < 123 = chr (ord x - 32)
    | otherwise = x

--
-- Sequences and series
--

-- | Arithmetic sequence
arithmeticSeq :: Double -> Double -> Int -> Double
arithmeticSeq a d n = a + (fromIntegral n) * d

-- | Geometric sequence
geometricSeq :: Double -> Double -> Int -> Double
geometricSeq a r n = a * r ** (fromIntegral n)

-- | Arithmetic series
arithmeticSeries :: Double -> Double -> Int -> Double
arithmeticSeries a d n = (fromIntegral n + 1) * (a + (d * fromIntegral n)/2)


-- | Geometric series
geometricSeries :: Double -> Double -> Int -> Double
geometricSeries a r n = a * (1- r ** (fromIntegral n+1))/(1-r)
```

2/3 -1 mark, you only need to use 2 'maxOf2' functions:
maxOf2 x (maxOf2 y z)

0.5/2 -½ mark for using 'ord', compare characters directly instead
-½ mark for magic numbers, use 'ord '0'' over 48
-½ mark, you don't need to pattern match on a boolean expression, instead return the expression itself

0.5/2, same as above

1.5/2 -½ mark, using magic numbers

3/4 -½ mark, using magic numbers
-½ mark, using ord instead of comparing characters directly

1/1

1.5/2, use ^ instead of **

1.5/2, you can factor out duplication of 'fromIntegral n' with a where clause

0.5/2 -½ mark, use '^' instead of '**'
-1 mark, when r is 1, you end up dividing by 0, you need to add a separate case for this

12/20

```
-------- Test Output --------
building the project
[1 of 3] Compiling IC.TestSuite     ( IC/TestSuite.hs, IC/TestSuite.o )
[2 of 3] Compiling Sequences        ( Sequences.hs, Sequences.o )
[3 of 3] Compiling Tests            ( Tests.hs, Tests.o )
running the tests
maxOf2: 3 / 3

maxOf3: 3 / 3

isADigit: 2 / 2

isAlpha: 2 / 2

digitToInt: 2 / 2

toUpper: 2 / 2

arithmeticSeq: 4 / 4

geometricSeq: 4 / 4

arithmeticSeries: 4 / 4

geometricSeries: 4 / 4


copying Tests.hs from skeleton
building the project
running the tests
maxOf2: 3 / 3

maxOf3: 3 / 3

isADigit: 2 / 2

isAlpha: 2 / 2

digitToInt: 2 / 2

toUpper: 2 / 2

arithmeticSeq: 4 / 4

geometricSeq: 4 / 4

arithmeticSeries: 4 / 4

geometricSeries: 4 / 4



-------- Test Errors --------

```